

Fuzzy Logic-Based Student Performance Evaluation System for COMSATS University

✓ 1. System Overview

The Fuzzy Inference System (FIS) for COMSATS University is designed to evaluate student academic performance by considering multiple qualitative and quantitative factors. Unlike traditional grading systems that use rigid thresholds, this system incorporates human-like reasoning to handle uncertainty and imprecision in student assessments.

2. Input Parameters & Linguistic Variables

The system evaluates students based on three key inputs:

A. Lecture Attendance (0-100%)

Fuzzy Set	Description	Range
Poor	Low attendance affecting learning	0-60%
Average	Moderate attendance with some gaps	40-90%
Excellent	Consistent, near-perfect attendance	80-100%

B. Assignment Scores (0-100%)

Fuzzy Set	Description	Range
Weak	Below-average performance	0-65%
Satisfactory	Meets expectations	50-90%
Strong	Outstanding work	80-100%

C. Class Participation (0-10 Scale)

Fuzzy Set	Description	Range
Low	Minimal engagement	0-4
Medium	Occasional contributions	2-8
High	Actively participates	6-10

3. Output: Performance Evaluation (0-100%)

The system generates a **final performance score** with the following categories:

Fuzzy Set	Description	Range
Needs Improvement	Below expectations	0-40%
Satisfactory	Meets minimum requirements	30-70%
Good	Above-average performance	60-90%
Excellent	Outstanding performance	80-100%

4. Fuzzy Rules (Decision Logic)

The system uses **IF-THEN** rules to map inputs to outputs. Examples:

1. **IF** (Attendance is **Poor**) **OR** (Assignments are **Weak**) **OR** (Participation is **Low**)
THEN Performance is **Needs Improvement**
(Indicates significant gaps requiring intervention)
2. **IF** (Attendance is **Average**) **AND** (Assignments are **Satisfactory**) **AND** (Participation is **Medium**)
THEN Performance is **Satisfactory**
(Meets baseline expectations)
3. **IF** (Attendance is **Excellent**) **AND** (Assignments are **Strong**) **AND** (Participation is **High**)
THEN Performance is **Excellent**
(Top-tier performance deserving recognition)
4. **IF** (Attendance is **Excellent**) **AND** (Assignments are **Strong**) **BUT** (Participation is **Medium**)
THEN Performance is **Good**
(Strong academically but could engage more in class)

5. Example Cases

Case 1: Weak Performer

- **Attendance:** 55% (**Poor**)
- **Assignments:** 60% (**Weak**)
- **Participation:** 3/10 (**Low**)
- **Evaluation Result:** Needs Improvement (25%)
- **Feedback:** "Requires academic support and better engagement."

Case 2: Average Student

- **Attendance:** 75% (**Average**)
- **Assignments:** 72% (**Satisfactory**)
- **Participation:** 6/10 (**Medium-High**)
- **Evaluation Result:** Satisfactory (65%)
- **Feedback:** "Meets expectations; could improve with more consistency."

Case 3: High Achiever

- **Attendance:** 92% (**Excellent**)
- **Assignments:** 88% (**Strong**)
- **Participation:** 9/10 (**High**)
- **Evaluation Result:** Excellent (89%)
- **Feedback:** "Outstanding performance across all metrics."

```
1 !pip install scikit-fuzzy
```

➡ Requirement already satisfied: scikit-fuzzy in /usr/local/lib/python3.11/dist-packages (0.5.0)

✓ =====IMPORT REQUIRED LIBRARIES =====

```
1
2 import numpy as np          # For numerical operations
3 import skfuzzy as fuzz     # For fuzzy logic functions
4 from skfuzzy import control as ctrl # For fuzzy control system
5 import matplotlib.pyplot as plt # For visualization (optional)
```

✓ =====FUZZIFICATION COMPONENT =====

In fuzzy logic control systems, **Antecedent** refers to an input variable that is used to evaluate rules and determine outputs.

In fuzzy logic control systems, a **Consequent** refers to the output variable that is determined based on the rules and input variables (Antecedents).

```
1
2
3 # Define the range and resolution of input/output variables
4
5 # 1. Attendance (0-100% in 1% increments)
6 attendance = ctrl.Antecedent(np.arange(0, 101, 1), 'attendance')
7
8 # 2. Assignment Scores (0-100% in 1% increments)
9 assignments = ctrl.Antecedent(np.arange(0, 101, 1), 'assignments')
10
11 # 3. Class Participation (0-10 scale in 0.1 increments)
```

```

12 participation = ctrl.Antecedent(np.arange(0, 11, 0.1), 'participation')
13
14 # 4. Performance Output (0-100% in 1% increments)
15 performance = ctrl.Consequent(np.arange(0, 101, 1), 'performance')

```

✓ ===== MEMBERSHIP FUNCTIONS =====

In fuzzy logic, **trimf** stands for Triangular Membership Function, which is a simple way to define how a crisp (numerical) input value maps to a fuzzy set (like "Low", "Medium", or "High").

```

1
2
3 # Define triangular membership functions for each variable
4
5 # Attendance Membership Functions
6 attendance['poor'] = fuzz.trimf(attendance.universe, [0, 0, 60])      # 0-60% is poor
7 attendance['average'] = fuzz.trimf(attendance.universe, [40, 70, 90]) # 40-90% is average
8 attendance['excellent'] = fuzz.trimf(attendance.universe, [80, 100, 100]) # 80-100% is excellent
9
10 # Assignment Membership Functions
11 assignments['weak'] = fuzz.trimf(assignments.universe, [0, 0, 65])    # 0-65% is weak
12 assignments['satisfactory'] = fuzz.trimf(assignments.universe, [50, 70, 90]) # 50-90% is satisfactory
13 assignments['strong'] = fuzz.trimf(assignments.universe, [80, 100, 100]) # 80-100% is strong
14
15 # Participation Membership Functions
16 participation['low'] = fuzz.trimf(participation.universe, [0, 0, 4])   # 0-4 is low
17 participation['medium'] = fuzz.trimf(participation.universe, [2, 5, 8]) # 2-8 is medium
18 participation['high'] = fuzz.trimf(participation.universe, [6, 10, 10]) # 6-10 is high
19
20 # Performance Membership Functions
21 performance['needs_improvement'] = fuzz.trimf(performance.universe, [0, 0, 40]) # 0-40%
22 performance['satisfactory'] = fuzz.trimf(performance.universe, [30, 50, 70])    # 30-70%

```

```
23 performance['good'] = fuzz.trimf(performance.universe, [60, 75, 90])      # 60-90%
24 performance['excellent'] = fuzz.trimf(performance.universe, [80, 100, 100]) # 80-100%
```

✓ ===== RULE BASE (INFERENCE ENGINE) =====

```
1 # Define the fuzzy rules that map inputs to outputs
2
3 rule1 = ctrl.Rule(
4     attendance['poor'] | assignments['weak'] | participation['low'],
5     performance['needs_improvement'],
6     "Rule 1: Any poor factor → Needs improvement"
7 )
8
9 rule2 = ctrl.Rule(
10    attendance['average'] & assignments['satisfactory'] & participation['medium'],
11    performance['satisfactory'],
12    "Rule 2: Average in all → Satisfactory"
13 )
14
15 rule3 = ctrl.Rule(
16    attendance['excellent'] & assignments['satisfactory'] & participation['high'],
17    performance['good'],
18    "Rule 3: Excellent attendance + satisfactory work + high participation → Good"
19 )
20
21 rule4 = ctrl.Rule(
22    attendance['excellent'] & assignments['strong'] & participation['high'],
23    performance['excellent'],
24    "Rule 4: Excellent in all → Excellent"
25 )
26
27 rule5 = ctrl.Rule(
```



```

28     attendance['average'] & assignments['weak'] & participation['medium'],
29     performance['needs_improvement'],
30     "Rule 5: Average attendance + weak assignments → Needs improvement"
31 )
32
33 rule6 = ctrl.Rule(
34     attendance['excellent'] & assignments['strong'] & participation['medium'],
35     performance['good'],
36     "Rule 6: Excellent with strong work but medium participation → Good"
37 )

```

✓ ===== CONTROL SYSTEM =====

```

1 # Combine all rules into a fuzzy control system
2 performance_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6])

```

✓ ===== SIMULATION ENVIRONMENT =====

```

1 # Create the simulation environment
2 performance_eval = ctrl.ControlSystemSimulation(performance_ctrl)
3

```

✓ ===== TEST CASE EVALUATION =====

```

1
2
3
4 def evaluate_student(att_score, assign_score, part_score):
5     """

```

```

6     Evaluates a student's performance based on given inputs
7     and returns the performance score with interpretation.
8     """
9     # Set input values
10    performance_eval.input['attendance'] = att_score
11    performance_eval.input['assignments'] = assign_score
12    performance_eval.input['participation'] = part_score
13
14    # Compute the result (fuzzification, inference, defuzzification)
15    performance_eval.compute()
16
17    # Get the crisp output value
18    perf_score = performance_eval.output['performance']
19
20    # Interpret the result
21    if perf_score < 40:
22        interpretation = "Needs Improvement"
23    elif perf_score < 70:
24        interpretation = "Satisfactory"
25    elif perf_score < 90:
26        interpretation = "Good"
27    else:
28        interpretation = "Excellent"
29
30    return perf_score, interpretation
31
32

```

✓ ===== DEMONSTRATION =====

```

1
2 if __name__ == "__main__":
3     # Example 1: Weak Performer

```

```
4     print("\nCase 1: Weak Performer")
5     score1, interp1 = evaluate_student(55, 60, 3)
6     print(f"Performance Score: {score1:.1f}% → {interp1}")
7
8     # Example 2: Average Student
9     print("\nCase 2: Average Student")
10    score2, interp2 = evaluate_student(75, 72, 6)
11    print(f"Performance Score: {score2:.1f}% → {interp2}")
12
13    # Example 3: High Achiever
14    print("\nCase 3: High Achiever")
15    score3, interp3 = evaluate_student(92, 88, 9)
16    print(f"Performance Score: {score3:.1f}% → {interp3}")
17
18    # (Optional) Visualize membership functions
19    if input("\nShow membership function plots? (y/n): ").lower() == 'y':
20        attendance.view()
21        assignments.view()
22        participation.view()
23        performance.view()
24        plt.show()
```



Case 1: Weak Performer

Performance Score: 35.8% → Needs Improvement

Case 2: Average Student

Performance Score: 50.0% → Satisfactory

Case 3: High Achiever

Performance Score: 86.8% → Good